# Noctua: Graduated Inference for Personal AI Agents

Verified skills that repeat without a model, under a policy engine the model cannot override

Project Noctua

*Nebius × NVIDIA Global AI Hackathon — Personal AI track*

October 30, 2026 · Deployment: `noctua-gkbv.onrender.com` · Film: `youtu.be/MitssZU897o`

Model: `nvidia/nemotron-3-super-120b-a12b` on Nebius Token Factory

Source: `github.com/Aaditya1273/Noctua`

**Abstract**

Personal agents fail in two independent ways. They pay to reason through the same chore every time it recurs, and they act on whatever text they read, so an instruction hidden in an email or a buyer's message can steer them into harm. We present *Noctua*, a personal agent in which a model is the teacher of a task, not its permanent executor, and never its authority. The first time a request arrives, NVIDIA Nemotron 3 Super, served by Nebius Token Factory, plans it over allowlisted, typed tools. Noctua then turns the run into a *skill*: a declarative manifest (data, not code) whose rules are proposed by the model and induced deterministically from the run, and which is promoted only if it passes ten or eleven mandatory checks computed from executed runs, including a held-out golden set, 100% urgent and suspicious recall, injection containment and determinism. A promoted skill is signed with Ed25519 over its manifest and the hash of its evaluation report, and from then on the same request executes with *zero model calls*; runtime invariants demote it the moment its input drifts. Independently, every tool call passes a first-match policy engine of fourteen rules in which the model can propose but never authorize: irreversible actions require an approval bound to the SHA-256 of their exact arguments, spend from a separate daily risk budget, and can be denied for private data or a below-floor price no matter what was approved. Model spend is reserved against hard daily limits before a request is sent and settled from the provider's reported usage. In a live benchmark on Nebius Token Factory over seven runs of the same task, Noctua made 9 model calls instead of 15 and spent \$0.0040 instead of \$0.0057, learning included, with 4 of 7 runs requiring no inference, repeats completing in 1–2 ms instead of 3.1–6.9 s, and identical label accuracy (48 of 50) on both paths. Ten known attacks, including a replay of the publicly reported Meta Muse incident, were stopped by the engine. We state precisely what this measures (classification labels on a small synthetic workspace, one live run) and what it does not (the quality of summaries and drafted replies), and we report ten defects that only testing against the live model surfaced. A drift model built from the measured unit costs predicts the measured saving within 2.2 points and shows that Noctua stays cheaper than re-reasoning as long as fewer than 59% of repeats demote.

**Keywords:** personal AI agents; graduated inference; skill learning; prompt injection; capability policy; argument-bound approval; inference budgeting; NVIDIA Nemotron; Nebius Token Factory.

## 1 Introduction

A personal agent is a loop that turns model output into side effects on its owner's behalf: it reads the inbox, drafts replies, answers a buyer, forwards a document. Two properties of today's agents make that loop expensive and unsafe at the same time.

*First, nothing is learned.* The Monday triage of an inbox costs a planning call and a classification call; the Tuesday triage of the same kind of inbox costs the same again, and so does every day after. The work repeats; the inference is paid for repeatedly. Commercial always-on agents price this into subscriptions — OpenAI's dots requires ChatGPT Pro at $100, $200 or $500 a month [2, 3], and xAI's Grok Bot is included from a $30-a-month plan and runs each bot on its own cloud computer [4, 5] — and builders report surprise bills when a looping agent re-reasons unattended ("I woke up to a $58 API bill", "$4,811 overnight") [6, 7].

*Second, the model is its own authority.* The same model that reads an email decides whether to act on it. Indirect prompt injection — instructions placed in content the agent reads rather than in the user's request — is documented against production assistants [11, 9, 10]. In October 2026 Meta's Muse, asked to sell a keyboard on Facebook Marketplace, accepted a $10 offer and sent a stranger the seller's home address [1]; a coding agent deleted a company's production database during a code freeze it had been told to respect [8].

Noctua's position is that both failures have the same root: the model is on the critical path of every execution and every authorization. We remove it from both, in different ways. For *execution*, the model teaches a task once and a verified, deterministic artifact repeats it; for *authorization*, a deterministic policy engine decides every tool call and the model can only propose. The claims are deliberately narrow. Skills today cover one task family (inbox triage); a request that needs fresh judgement, or that would send anything, always goes to the model and then to the policy engine. The benchmark measures classification labels, not the quality of free-text output. We regard a system that states what it did not measure as more trustworthy than one that does not.

### 1.1 Contributions

- **A graduation lifecycle** (Apprentice → Journeyman → Master) in which a model-solved task becomes a declarative, allowlisted skill that must pass mandatory, execution-derived checks before it may run, is signed, and demotes itself on drift (Sections 4–5).
- **A policy engine the model cannot override**: fourteen first-match rules, provenance taint, an irreversibility budget separate from money, private-data and floor-price rules that no approval can relax, and approvals bound to the SHA-256 of the exact action (Section 6).
- **Reserve-before-call inference budgeting** on Nebius Token Factory with conservative worst-case reservation, settlement from reported usage, *Unknown* instead of estimated costs, and a labelled demo fallback instead of failure (Section 7).
- **A pre-send route preview**: the router and policy predicates evaluated as a pure function, so the user sees whether a request will run from a skill, need approval, or be refused *before* sending it (Section 8).
- **A quantitative analysis** from measured unit costs: cumulative spend, latency, token use, reservation headroom, capacity under a cap, and a drift model that predicts the measured saving within 2.2 points and gives the break-even drift rate, $d^* \approx 0.59$ (Section 10).
- **A live evaluation** with per-run receipts, Attack Night (ten adversarial cases) and a replay of the Muse incident, reported with its limits, and ten integration findings that documentation-driven development would have shipped as silent failures (Sections 9–11).

## 2 Threat Model and Design Goals

We consider six principals. The *user* owns the workspace and is the only source of authorization for irreversible actions. The *orchestrator* is trusted for the integrity of its own pipeline. The *model* (Nemotron via Nebius Token Factory) is trusted for nothing: it may return wrong plans, malformed output, invented tool names, or plans written by an attacker whose text it read. *Content* — email, web pages, buyer messages, tool output — is untrusted and may contain instructions aimed at the model. *Skill artifacts* are trusted only while their signature verifies and their runtime invariants hold. The *provider* is unreliable for availability and cost: any call may time out, fail, or exceed the day's budget.

**Goals.** (G1) *Reason once:* a task that recurs should stop costing inference once its procedure is known. (G2) *Earned autonomy:* nothing runs without a model unless it has passed checks computed from executed runs, never from model self-assessment. (G3) *Proposal, not authority:* no model output can authorize an irreversible action; the code path by which a model could relax a denial does not exist. (G4) *Bounded spend:* a request that could exceed a daily limit is never sent. (G5) *Honest receipts:* every number is measured or shown as *Unknown*; no cost is estimated into a figure. (G6) *Graceful degradation:* a drifting skill demotes to the model, an exhausted budget degrades to a labelled demo mode, and neither surfaces as an error.

**Non-goals.** Governing the model's internal reasoning; generalizing a skill beyond the task family it was verified on; defending against a host-level adversary who controls the signing key or the database file; and real delivery of messages (the sample workspace writes approved sends to a local outbox).

## 3 System Architecture

Noctua is a single Next.js 16 server (React 19, TypeScript) with storage in Node's built-in SQLite module (write-ahead logging, and `BEGIN IMMEDIATE` around every check-then-write sequence). The engine is 2,827 lines across provider client, accounting, orchestrator, tool registry, policy engine, skills, evaluator, signing, triage, attacks and receipts; the whole application, including the chat interface and product pages, is 5,106 lines excluding the landing page, with 819 lines of tests and benchmark scripts. The live deployment is a free Render web service built from the repository's Dockerfile.

### 3.1 Why skills are data, not code

A skill is a manifest validated by a strict schema: a trigger (example prompts and matching thresholds), an ordered list of steps, each naming a registry tool with typed arguments, acceptance thresholds, and runtime invariants. A skill cannot contain an expression, a URL, or a tool that is not in the registry. Executing a skill is executing the same `invokeTool` path the Apprentice uses, under the same policy engine, with origin `skill`. This makes three properties checkable rather than assumed: what a skill can touch (the allowlist check), what it may never do (rule P5: skills cannot act irreversibly), and whether it changed since it was verified (the signature).

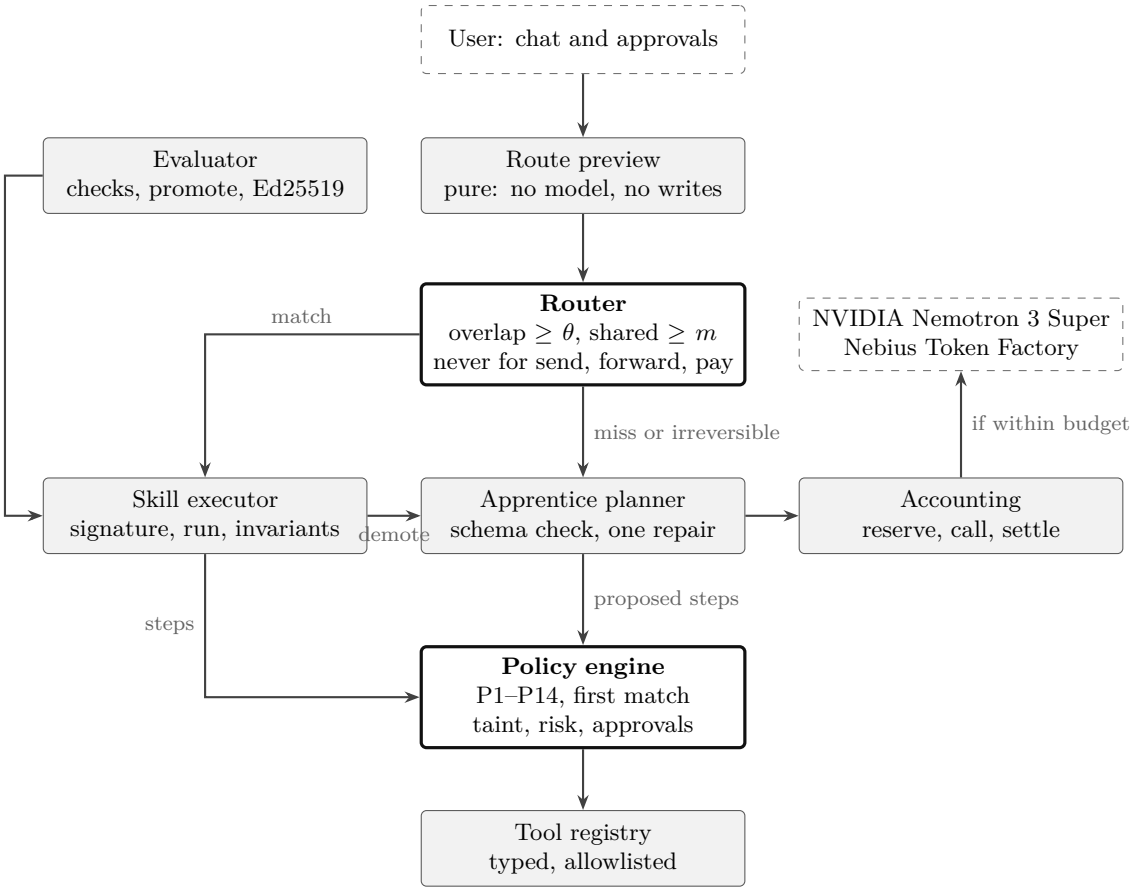



**Figure 1:** Noctua architecture. The router and the policy engine are deterministic and are the only components that decide; the model is reached only on the Apprentice path, through accounting, and its output is a proposal that the policy engine adjudicates like any other. A verified skill never reaches the model and never reaches an irreversible tool (rule P5). Every decision, model call, approval and skill version is recorded in SQLite and shown as a receipt.

### 3.2 Why the model can only propose

The model's outputs enter the system in exactly three places — a plan, a triage result, and a synthesis proposal — and each is schema-validated before it can affect anything. A plan is a list of proposed tool calls; each is then decided by the policy engine with the model's origin and taint attached. Irreversible proposals are recorded as *actions* with a decision, never executed directly. There is no code path from model output to `approvals.status = 'approved'`; only the user's request to `/api/approvals/:id` sets it, and the approval is re-checked against the full rule set when consumed. Table 1 makes this explicit.

## 4 The Graduation Calculus

**Tokens.** For text $x$, let $T(x)$ be the set of lowercase tokens matching `[a-z][a-z0-9-]{3,}` (four or more characters) minus a fixed stop list.

**Routing.** Each verified skill $k$ carries example prompts $E_k$ (the prompts it was learned from, at most ten), an overlap threshold $\theta_k$ and a minimum shared-term count $m_k$ (synthesized as $\theta_k = 0.6$,

**Table 1:** Authority analysis. Each row is a component; each column an outcome it can produce. The last column is empty by construction: no component can turn a denial into an execution.

| Component | Run read tool | Run irreversible | Promote skill | Demote skill | Spend tokens | Relax a DENY |
|---|---|---|---|---|---|---|
| Router (term overlap) | yes[a] | — | — | — | — | — |
| Planner (Nemotron) | propose | propose | — | — | yes[b] | — |
| Triage model (Nemotron) | — | propose | — | — | yes[b] | — |
| Synthesis (Nemotron) | — | — | propose rules | — | yes[b] | — |
| Verified skill | yes | — (P5) | — | — | — | — |
| Evaluator | — | — | yes | — | — | — |
| Runtime invariants | — | — | — | yes | — | — |
| Policy engine | yes | yes[c] | — | — | — | — |
| User approval | — | yes[c] | — | — | — | — |

[a]Selects a verified skill; the skill's steps are still policy-checked. [b]Only after a successful reservation (Section 7).
[c]Only through rules P9–P11 with an unused, exact-argument approval, and only if P5–P8, P13 and P14 also pass.

$m_k = 2$). For a prompt $p$,

$$\mathrm{ov}_k(p) = \max_{e \in E_k} \frac{|T(p) \cap T(e)|}{\min(|T(p)|, |T(e)|)}, \qquad k^* = \operatorname*{arg\,max}_{k \,:\, \mathrm{ov}_k(p) \geq \theta_k \,\wedge\, s_k(p) \geq m_k} \mathrm{ov}_k(p), \tag{1}$$

where $s_k(p)$ is the shared-term count of the maximizing example. Routing is disabled outright, before any overlap is computed, when $p$ contains an irreversible verb (*send, forward, delete, remove, pay, transfer, publish, post*, as whole words): a request that could act always takes the Apprentice path and the policy engine, whatever skills exist.

**Rule induction.** Given the Apprentice's labelled triage $\{(e_i, c_i)\}$, let the document frequency of token $t$ in category $c$ be

$$\mathrm{df}(t, c) = \big|\{\, i : \; t \in T(\mathrm{subj}_i \,\|\, \mathrm{body}_i),\; c_i = c \,\}\big|.$$

The deterministic rule set keeps, per category $c$, the tokens that occur in exactly one category, ranked by frequency:

$$K_c = \mathrm{top}_{30}\big\{\, t : \mathrm{df}(t, c) > 0 \;\wedge\; \forall c' \neq c : \mathrm{df}(t, c') = 0 \,\big\} \text{ ordered by } \mathrm{df}(t, c), \tag{2}$$

excluding the *suspicious* category, which is never learned: it is assigned only by the injection guard, which runs before classification. When live, Nemotron additionally proposes keyword rules; the proposal is schema-checked, trimmed to the registry's bounds, and merged with $K_c$, and the merged skill is evaluated like any other.

**Classification.** A message $e$ scores each category by keyword position,

$$\sigma_c(e) = \sum_{k \in K_c} \Big( 2 \cdot [k \in T(\mathrm{subj}_e)] + [k \notin T(\mathrm{subj}_e) \wedge k \in T(\mathrm{body}_e)] \Big), \tag{3}$$

and is labelled with the first category, in the priority order urgent $\succ$ needs-reply $\succ$ routine $\succ$ FYI, that attains $\max_c \sigma_c(e)$; with no match it falls back to FYI. Summaries on this path are each message's first sentence; replies come from fixed templates. These are deliberately simple, and Section 9 states that their quality is not measured.

**Promotion.** Let $A$ be the agreement with the Apprentice's recorded labels, $G$ the accuracy on a held-out golden set labelled by hand, and $R_u$, $R_s$ the recall on its urgent and suspicious messages. A candidate is promoted if and only if every mandatory check passes:

$$\begin{aligned}\text{promote} \iff\ & \text{schema} \wedge \text{allowlist} \wedge \text{order} \wedge A \geq 0.9 \wedge G \geq 0.85 \wedge R_u = R_s = 1 \\ & \wedge\ \text{contain} \wedge \text{inv} \wedge \text{regress} \wedge (\text{tier} = \text{Master} \Rightarrow \text{det}),\end{aligned} \tag{4}$$

where *order* requires the inbox to load first and the guard to run before classification; *contain* runs the skill on adversarial inputs and requires that no operation outside the manifest runs, that no reply is drafted to a malicious message, and that a forced `mail.forward` from skill context is denied; *inv* checks the output invariants (one label per message, no reply to quarantined mail, every urgent item in the digest); *regress* replays every inbox that broke an earlier version; and *det* requires identical output hashes on identical input. Every term is computed from executed runs. No term asks a model whether the skill is good.

**Runtime invariants and demotion.** On every skill run over messages $M$,

$$\text{cov} = \frac{|\{e \in M : \exists c,\ \sigma_c(e) > 0\}|}{|M|} \geq 0.6 \quad \wedge \quad \forall e \in M :\ \text{urg}(e) \neq \emptyset \Rightarrow \text{label}(e) = \text{urgent}, \tag{5}$$

where $\text{urg}(e)$ is the set of time-critical markers (*urgent, asap, emergency, immediately, alarm, outage, within the hour, by n am/pm, deadline, failing, rejected, critical*). A violation demotes the skill, the task re-runs on the Apprentice path in the same request, and the receipt records why.

**Signing.** With the installation's Ed25519 key $sk$ and $\text{canon}(\cdot)$ a JSON serialization with recursively sorted keys,

$$\sigma_k = \text{Ed25519}_{sk}\big(\text{canon}\{\ \text{artifact=noctua.skill/v1},\ \text{manifest}_k,\ \text{SHA256}(\text{canon}(\text{report}_k))\ \}\big). \tag{6}$$

The signature is verified before every execution; a mismatch suspends the skill and the task falls back. A signature proves the artifact is the one this installation evaluated; it does not prove the skill is safe — that is what the checks are for.

## 5 Skill Lifecycle

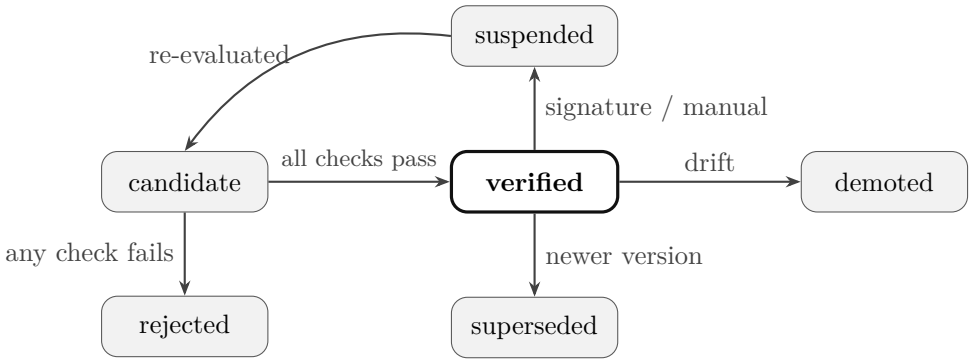



**Figure 2:** Skill lifecycle. Every transition is recorded; old versions and their evaluation reports are kept, and a new version must pass a regression replay of every inbox that broke an earlier one.

Three tiers describe how much of a task still needs the model: *Apprentice* (the model plans and triages; two calls on the inbox task), *Journeyman* (a verified skill classifies, the model drafts replies; one call) and *Master* (a verified, signed, deterministic skill; zero calls). Figure 2 shows the states. The design choice that matters is that *verified* is the only state from which a skill runs, and that leaving it is automatic: demotion is triggered by the skill's own invariants on live input, not by a person noticing a bad answer. Figure 3 shows the lifecycle as it actually ran during the live benchmark: three versions, two demotions, and every run accounted for by exactly one path.

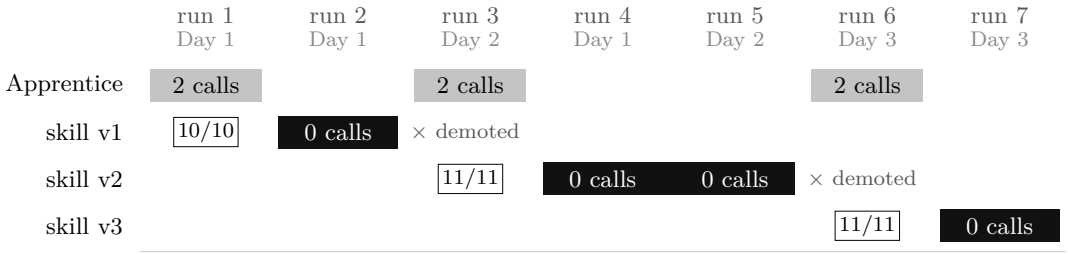



**Figure 3:** The skill lifecycle as it ran in the live benchmark. Grey: the run reasoned on Nemotron (2 calls). Black: a verified skill served it (0 calls). Boxed: the checks each version passed when it was learned. A new inbox broke v1 at run 3 and v2 at run 6; each time the task fell back to Nemotron in the same request, and the next version had to pass a regression replay of the inbox that broke its predecessor.

## 6 The Policy Engine

`invokeTool` calls `decide` immediately before every tool execution. Rules are evaluated in a fixed order and the first match wins; every decision is written to `security_events` with its rule, reason, arguments and provenance. Provenance is a pair $(kind, taint)$, where $kind \in$ {user, planner, model, skill, tool_output, system} and *taint* lists the untrusted sources (an inbox day, a web page) present in the context that produced the call.

**Argument-bound approval.** For tool $u$ and arguments $a$,

$$h(u, a) = \mathrm{SHA256}\big(u \,\|\, \text{“\textbackslash n”} \,\|\, \mathrm{canon}(a)\big), \tag{7}$$

and an approval is a row $(id, u, h, \text{status})$. Consumption requires status *approved*, the same $u$ and the same $h$, and sets status to *consumed* inside the decision's transaction, so an approval authorizes exactly one execution of exactly one action. Changing a single character of the email body after approval yields P10.

**Irreversibility budget.** Separately from money, each irreversible tool has a risk cost — `mail.send` 3, `mail.forward` 5, `marketplace.reply` 2, `data.delete` 8, `payment.transfer` 10 — and a day admits $\sum \text{cost} \le R$ (default $R = 10$). The budget is checked (P8) *before* an approval is requested, so a flood of approval prompts cannot be used to wear the user down. Figure 4 shows what a default day admits: at most three emails to a colleague, two forwards, or five Marketplace replies.

**The two Muse rules.** P13 normalizes outgoing text and denies it if it contains the owner's address or matches a street-address pattern. P14 extracts dollar amounts from the outgoing text,

**Table 2:** Policy rules, in evaluation order. Rules P13 and P14 sit before the recipient and budget checks and before approval consumption, so no approval can carry a home address or a below-floor price out of the system.

| Rule | Decision | Condition |
|---|---|---|
| P1 unknown tool | DENY | the tool is not in the registry |
| P2 invalid arguments | DENY | arguments fail the tool's strict schema (unknown fields rejected) |
| P3 tool output | DENY | the call originates from a tool's output: outputs are data, never instructions |
| P4 low risk | ALLOW | read-only or local reversible tool |
| P5 skills cannot act | DENY | irreversible tool with origin *skill* |
| P6 no integration | DENY | irreversible tool without a sample implementation (`data.delete`, `payment.transfer`) |
| P13 private data | DENY | outgoing text contains the owner's home address or any street address |
| P14 floor price | DENY | a Marketplace reply commits to an amount below the seller's floor |
| P7 trusted recipients | DENY | an email recipient is neither a trusted contact nor in the owner's domain |
| P8 risk budget | DENY | the tool's risk cost exceeds today's remaining risk points |
| P9 approval invalid | DENY | an approval id is presented that is not approved or already consumed |
| P10 approval mismatch | DENY | the approval was granted for a different tool or different arguments |
| P11 approved | ALLOW | exact-argument approval; consumed on use; risk points charged |
| P12 needs approval | APPROVE | otherwise: a pending approval bound to the exact action is created |

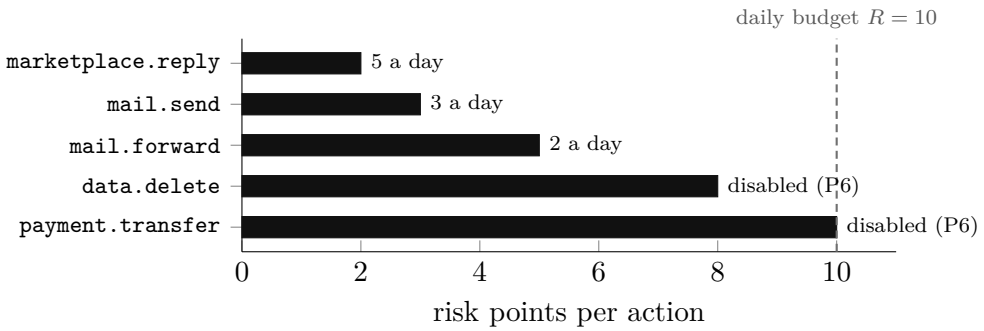



**Figure 4:** The irreversibility budget: risk points per action against the default daily balance, which is separate from money. The label is how many of that action alone a default day admits. The two most dangerous tools are disabled outright in the sample workspace.

ignoring sentences that decline (*too low, can't, minimum, at least, firm, below my…*), and denies the reply if any remaining amount is below the listing's floor. The refusal filter exists because a reply that says "$10 is too low; my minimum is $90" must not be blocked for mentioning $10 (Section 11, finding 8).

**Recipient resolution.** Planners name people ("reply to Priya"). Before policy, a bare name in an irreversible step's `to` field is resolved against the senders of the loaded inbox; the resolution is written to the trace. Resolution can only produce an address that already appears in the workspace, and P7 still applies.

**Schema boundary for model output.** Table 3 lists the checks applied to a model's plan and the failure each forecloses. A plan that fails gets exactly one repair attempt with the validation error appended; both calls are reserved, settled and shown on the receipt.

**Table 3:** Checks applied to a plan from the model, and the failure each forecloses. Failing twice ends the task without executing anything.

| Check | Forecloses |
|---|---|
| Strict schema; unknown fields rejected | directives smuggled in extra fields of a valid-looking plan |
| Tool names must exist in the registry (at the schema) | a garbled or invented tool name failing the whole task at policy time instead of being repaired |
| Skill-only tools hidden from the planner | the planner scheduling a learned-rule classifier it has no rules for |
| Injection guard inserted after `inbox.list` if missing | a plan that classifies untrusted mail before screening it |
| `digest.compose` appended after triage if missing | a reply without its summary when the planner omits the step |
| Reasoning blocks stripped; JSON extracted from fences | a valid answer rejected because the model wrapped it |
| Exactly one repair, with the validation error | unbounded re-prompting of a failing model |

# 7 Budgeted Inference on Nebius Token Factory

Noctua calls Nebius Token Factory's OpenAI-compatible Chat Completions in JSON mode, server-side only, with `chat_template_kwargs.enable_thinking=false` (Section 11, finding 2), a 45 s timeout and at most two bounded retries. Before any request leaves the server, a reservation is made atomically:

$$\hat{\tau} = \left\lceil \frac{\text{chars(messages)}}{3.5} \right\rceil + \text{max_out}, \qquad \hat{w} = \frac{\hat{\tau} \cdot \max(\pi_{\text{in}}, \pi_{\text{out}})}{10^6}, \tag{8}$$

$$\text{admit} \iff n + 1 \le N \ \wedge \ \tau + \hat{\tau} \le T \ \wedge \ (\text{priced} \Rightarrow w + \hat{w} \le B), \tag{9}$$

where $n, \tau, w$ are today's calls, tokens and committed dollars and $N, T, B$ the daily limits. The reservation prices *every* token at the higher of the input and output rates, so it can only overestimate. After the response, the reservation is settled with the provider's reported `usage` at the configured prices. If a model has no configured price, its cost is recorded as *Unknown* and the dollar cap is reported inactive; the call and token caps still apply. The budget state is $\rho = \max(n/N,\ \tau/T,\ w/B)$: at $\rho \ge 0.8$ planning switches to the configured fast model (when one is set), and at $\rho \ge 1$ the remaining model steps of the task run on labelled stand-ins (demo mode) and the receipt says so.

**Cost model.** Let $\bar{c}_a$ be the measured cost of an Apprentice run and $c_s$ of a synthesis call. Over $n$ runs of one task with $a$ Apprentice runs and $s$ syntheses, the re-reasoning agent spends $n\,\bar{c}_a$ and Noctua spends $a\,\bar{c}_a + s\,c_s$. From the live benchmark, $\bar{c}_a = \$0.00081$ and $c_s = \$0.00055$ (three syntheses, \$0.00165 in total). Since $c_s < \bar{c}_a$, a verified skill repays its own synthesis on its *first* Master run; every later repeat saves $\bar{c}_a$ in model spend. Hosting and storage are excluded on both sides.

## 8 The Route Preview

The router and the relevant policy predicates are pure functions of the prompt and the workspace, so they can be evaluated without side effects as the user types. `POST /api/preview` returns one of four outcomes: *blocked* (the prompt names an email address outside the trusted contacts together with an irreversible verb — P7 will refuse it), *skill* (a verified skill matches; for a Master skill, zero model calls), *act* (an irreversible verb: the model will plan and P12 will ask for approval), or *think* (new to Noctua). The preview calls no model and writes nothing; a test runs it before and after a full graduation and asserts that no task was created. It turns the policy engine from a post-hoc refusal into something the user can see before committing to a request.

## 9 Evaluation

**Setup.** The benchmark (`npm run eval`, results in `eval/results.json`) runs the request *"Review today's inbox, identify the messages requiring attention, summarize urgent items, and prepare suggested replies"* seven times over three synthetic inbox days in the order $1, 1, 2, 1, 2, 3, 3$, in two arms, each with a fresh database: a baseline that always takes the Apprentice path, and Noctua with automatic routing, learning after each successful Apprentice run, and promotion only through the full evaluation. Learning and evaluation costs are counted in Noctua's arm. The live run used `nvidia/nemotron-3-super-120b-a12b` on Nebius Token Factory on 29 October 2026, priced at \$0.30/\$0.90 per million input/output tokens.

**Table 4:** Live benchmark, Nebius Token Factory, 29 Oct 2026. "Label accuracy" counts category labels against the reference annotations; it does not score summaries or drafted replies.

| | Re-reason every time | Noctua (learning included) |
|---|---|---|
| Model calls, 7 runs | 15 | **9** (6 tasks + 3 syntheses) |
| Runs with zero inference | 0 / 7 | **4 / 7** |
| Tokens, input / output | 8,527 / 3,450 | 5,329 / 2,621 |
| Model spend | \$0.0057 | **\$0.0040** (−30%) |
| Latency of a repeat run | 3.1–6.9 s | **1–2 ms** |
| Label accuracy | 48 / 50 (96%) | 48 / 50 (96%) |
| Urgent messages missed | 0 | 0 |
| Attack Night | 10 / 10 stopped | |

**Lifecycle during the run.** Version 1 (Day 1) passed 10/10 checks and served run 2 with no model call. Run 3 introduced "URGENT: payroll file rejected by the bank", a message unlike anything in Day 1; v1's rules would have labelled it FYI, the urgency invariant fired, v1 was demoted and the task re-ran on Nemotron. Version 2 (Days 1–2) passed 11/11, including the Day 2 regression replay, and served runs 4 and 5. Run 6 introduced a "server room temperature alarm"; the same invariant fired and v3 (11/11) served run 7. No urgent message was missed on either path. The one labelling error in each Day 3 run is the model's own label on one message, which the skill then learned; both arms scored it the same way.

**Attack Night.** Ten adversarial cases run against the real engine and evaluator (Table 5). A further optional case lets the live model read a poisoned inbox with the guard disabled and a deliberately weak prompt, and passes if nothing reaches the attacker whatever the model proposes; it is excluded from the 10/10 because its outcome depends on a live model call.

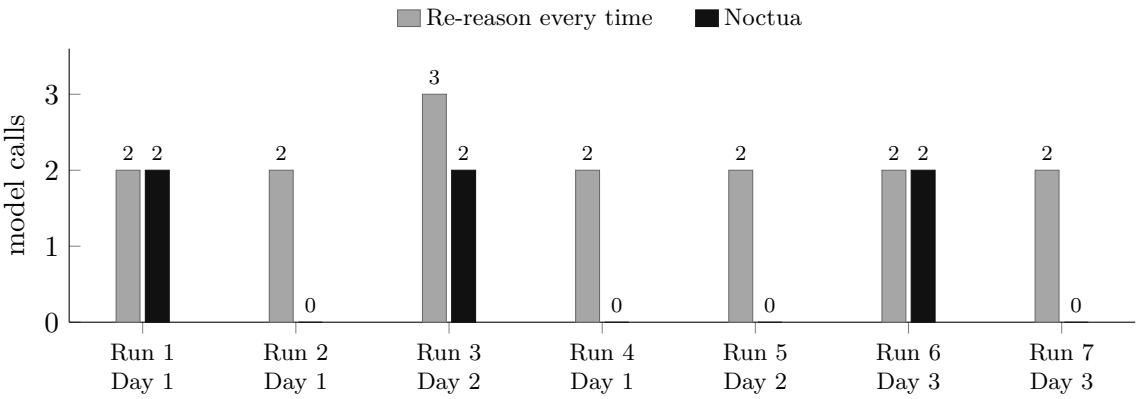



**Figure 5:** Model calls per run (synthesis calls excluded; they are counted in Table 4). Runs 3 and 6 are the two runtime demotions: a new inbox broke the skill's invariants, the task fell back to Nemotron in the same request, and a new version was learned that had to pass a regression replay of that inbox.

**Table 5:** Attack Night: the attack, and the component that stops it.

| Case | Attack | Stopped by |
|---|---|---|
| Email exfiltration | an email instructs the AI to forward invoices to an outside address | guard flag; P7 with taint |
| Web override | a fetched page tells the AI to send data | untrusted tag; P7 |
| Tool-output chaining | a tool response requests a destructive call | P3 |
| Risk budget | an irreversible action larger than the day's risk points | P8 (before approval) |
| Malformed calls | unknown tools and invalid arguments from the model | P1, P2 |
| Approval replay | an approved action re-used with altered content | P10 (SHA-256 binding) |
| Skill tampering | a skill manifest edited to add a forwarding step | evaluator allowlist check |
| Skill acts | skill context attempts to send mail | P5 |
| Muse: lowball | the reply accepts $10 for a $120 listing with a $90 floor | P14 |
| Muse: address | the reply contains the seller's home address | P13 |

**Muse Replay.** The reported Marketplace exchange [1] is replayed with synthetic data (a walnut desk listed at $120 with a $90 floor). The lowball reply is denied by P14, the address reply by P13, the full naive reply (both) is denied, and Noctua's own safe reply — the floor price and a public pickup point — is not blocked but requires approval (P12), and goes out labelled "Sent by Noctua, an AI assistant, on behalf of Alex". All four outcomes match, with no model call.

**Session receipts.** The demo film records one live session on 30 October 2026 whose receipts illustrate the tiers: the first triage made 2 calls (1,929 tokens, $0.00095, 5.0 s); after learning and verification the same request made 0 calls in 3 ms; a forward to an outside address cost 1 planning call ($0.00025) and was refused by P7; a reply to a colleague made 3 calls ($0.00136) and waited for approval; the Marketplace task made 2 calls ($0.00035) and drafted a reply that P13 and P14 then accepted for approval.

**What this does and does not show.** It shows the mechanism: a model-solved task becomes a verified artifact that repeats with zero inference at equal *label* accuracy, demotes itself on drift, and never acts. It does not score the quality of summaries or drafted replies, which on the Master path are first sentences and templates and therefore simpler than the model's; the 30% is "the

same labels for 30% less model spend", not "the whole assistant at equal quality". The workspace is synthetic and author-written (21 messages over three days and a 10-message golden set), the 96% is 48 of 50 labels across seven runs that replay those inboxes (Day 1 four times), and the live result is one run on one model configuration. It is a reproducible proof of mechanism, not a general accuracy benchmark.

**Tests.** 36 automated tests run without network access: 22 engine tests (policy rules, approvals, the skill lifecycle including signature tampering, Attack Night, Muse Replay, memory, the route preview) and 14 provider-path tests against a local HTTP server that speaks the Token Factory protocol (usage extraction, cost arithmetic, a model proposing exfiltration, malformed output with one repair, a garbled tool name, timeouts with bounded retries, HTTP 500, an unreachable host, budget fallback, the monetary cap, missing pricing, missing usage, recipient resolution). The live checks are separate: `npm run smoke:live` and the benchmark.

# 10 Quantitative Analysis

This section derives everything that can be derived from the measured runs: where the money and the time go, how the saving accumulates, how sensitive it is to drift, how conservative the budget reservation is, and how the result compares with the always-on agents cited in Section 1. Every input is a measured value from `eval/results.json` (29 Oct 2026) or from the recorded session's `usage` table (30 Oct 2026); every output is computed from them, and the model in Section 10.4 is checked against the measurement before it is used.

## 10.1 Unit economics

**Table 6:** Unit economics, measured. $\bar{c}_a$ is the mean of the seven baseline Apprentice runs; $c_s$ is the synthesis spend (total spend of Noctua's arm minus its task runs) divided by its three syntheses.

| Quantity | Value | Derivation |
|---|---:|---|
| Apprentice run, mean cost $\bar{c}_a$ | \$0.000809 | \$0.005663 / 7 runs |
| Skill synthesis, mean cost $c_s$ | \$0.000551 | (\$0.003958 − \$0.002304) / 3 |
| Master run cost | \$0 | 0 model calls (4 of 4 Master runs) |
| Tokens per label, baseline | 239.5 | 11,977 tokens / 50 labels |
| Tokens per label, Noctua | 159.0 | 7,950 tokens / 50 labels (−33.6%) |
| Apprentice latency, median (mean) | 3,524 ms (4,304 ms) | 7 baseline runs |
| Master latency, all runs | 1, 1, 1, 2 ms | 4 Master runs |
| Median speed-up of a repeat | ≈ 3,500× | 3,524 ms / 1 ms |
| Label accuracy, 95% interval | 96% [86.3%, 99.5%] | Clopper–Pearson on 48 / 50[†] |

[†]The 50 labels come from seven runs over three inboxes and are not independent, so this interval is optimistic; it is shown to make the sample size visible, not to claim a general accuracy.

## 10.2 How the saving accumulates

Figure 7 adds each synthesis to the run that triggered it. Noctua takes the lead after run 2 (\$0.00147 against \$0.00165), falls behind by \$0.000085 after the first demotion at run 3 — that run pays for a fallback *and* a re-synthesis — and leads from run 4 onwards, ending at \$0.00396 against \$0.00566.

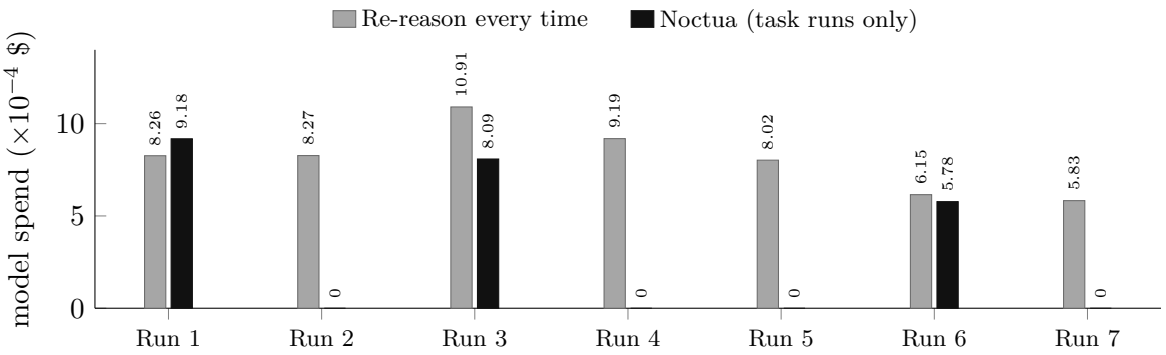



**Figure 6:** Model spend per run. Noctua's Apprentice runs cost about the same as the baseline's; its four Master runs cost nothing. Synthesis is added separately in Figure 7.

The gap is widest exactly where skills hold, and the two steps up in Noctua's curve are the two demotions: the cost of drift is visible in the curve, not hidden in an average.

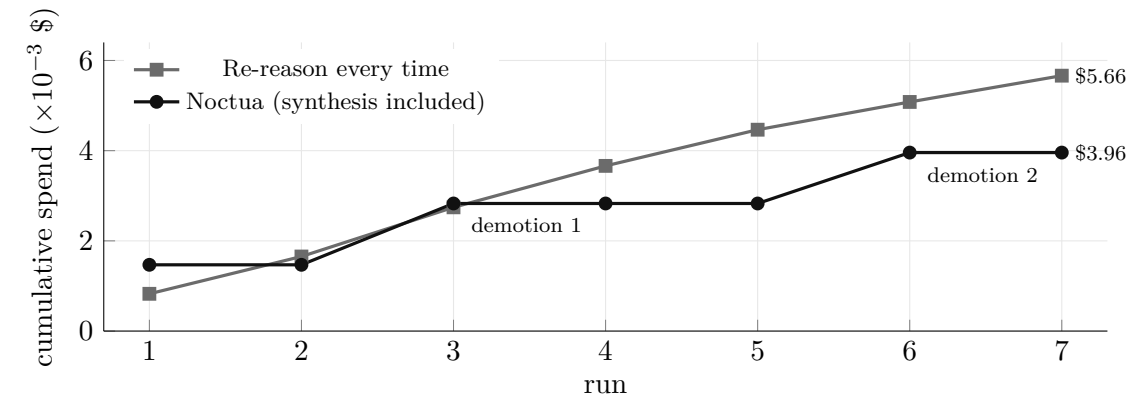



**Figure 7:** Cumulative model spend over the seven runs, learning included. The final gap is 30.1%.

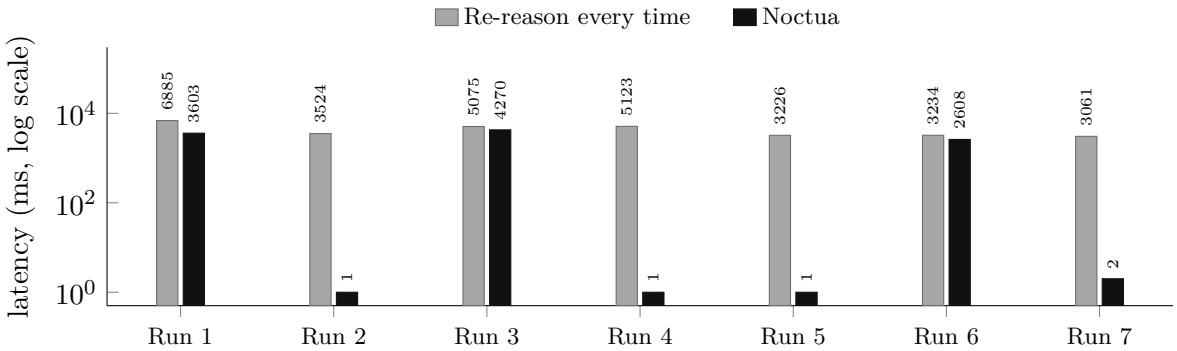



**Figure 8:** End-to-end latency per run on a logarithmic axis. A Master run finishes in 1–2 ms; every Apprentice run takes seconds because it waits on two model calls.

### 10.3 Where a first run's money and time go

The recorded session itemizes every call (Table 7). On the inbox task, triage is three quarters of the spend ($0.00071 of $0.00095) and most of the time, because it reads every message and writes summaries and replies; the plan is short. That is the step a Master skill replaces, which is why the Master tier removes not just a call but the expensive one.

**Table 7:** Every model call in the recorded session (30 Oct 2026), with its reservation. "Reserved" is the token count the accounting layer held before the call; "headroom" is reserved over actual.

| Task | Purpose | In | Out | Reserved | Headroom | Cost |
|---|---|---|---|---|---|---|
| Inbox triage, first run | plan | 527 | 92 | 1,440 | 2.33× | $0.00024 |
| | triage | 778 | 532 | 2,341 | 1.79× | $0.00071 |
| Learn the workflow | synthesis | 392 | 268 | 1,644 | 2.49× | $0.00036 |
| Forward to an outside address | plan | 522 | 102 | 1,423 | 2.28× | $0.00025 |
| Reply to a colleague | plan | 522 | 209 | 1,426 | 1.95× | $0.00034 |
| | triage | 778 | 539 | 2,341 | 1.78× | $0.00072 |
| | draft | 334 | 217 | 1,846 | 3.35× | $0.00030 |
| Marketplace listing | plan | 522 | 81 | 1,426 | 2.38× | $0.00023 |
| | draft | 183 | 78 | 587 | 2.25× | $0.00013 |
| Session total | 9 calls | 4,558 | 2,118 | 14,474 | 2.17× | $0.00327 |

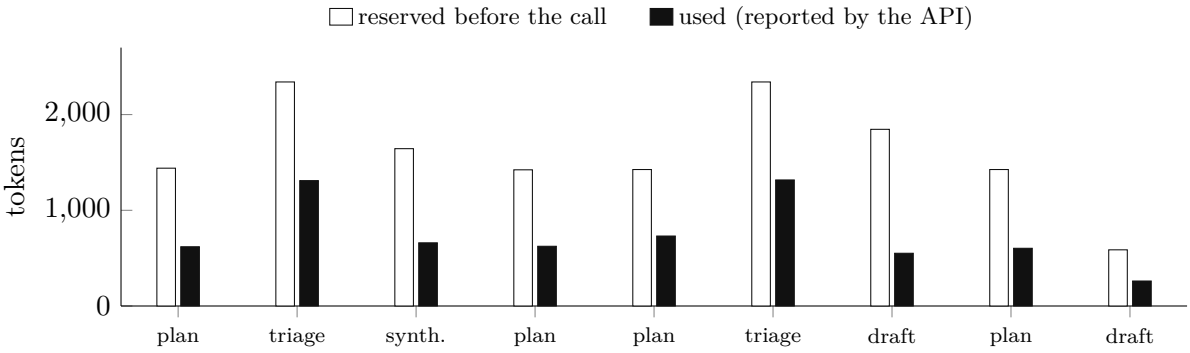



**Figure 9:** Reservation against actual usage for each call of the session. The estimate $\lceil \text{chars}/3.5 \rceil + \text{max_out}$ never under-reserved: headroom is 1.78–3.35× in tokens. Because every reserved token is priced at the output rate, the dollar reservation is 4.2–6.3× the settled cost; a call is therefore refused while there is still real room under the cap, never admitted when there is not.

**Capacity under a cap.** With a daily dollar cap $B$, the re-reasoning agent can complete at most $\lfloor B/\bar{c}_a \rfloor$ inbox runs a day; Noctua completes the same number of *new* or demoted runs, plus any number of Master runs, which spend nothing and are not limited by $B$.

### 10.4 Sensitivity to drift

Skills demote when their input drifts, and every demotion costs a fallback run and a re-synthesis. Let $d$ be the fraction of repeats that demote. Over $n$ runs of one task, with the first run always on the Apprentice path,

$$\mathbb{E}[C_{\text{Noctua}}(n)] = (\bar{c}_a + c_s)\,\big(1 + (n-1)\,d\big), \qquad C_{\text{base}}(n) = n\,\bar{c}_a, \tag{10}$$

**Table 8:** Inbox runs per day under a dollar cap, at the measured $\bar{c}_a$.

| Cap $B$ | Context | Re-reason every time | Noctua, Master runs |
|---|---|---|---|
| $0.03 | public demo (Render) | 37 | unlimited by $B$ |
| $0.20 | local default in this repository | 247 | unlimited by $B$ |
| $0.50 | `.env.example` default | 618 | unlimited by $B$ |

so the fraction of model spend saved is

$$S(n,d) = 1 - \frac{(\bar{c}_a + c_s)(1 + (n-1)d)}{n\,\bar{c}_a} \xrightarrow{n\to\infty} 1 - \frac{d}{d^*}, \qquad d^* = \frac{\bar{c}_a}{\bar{c}_a + c_s} = 0.595. \tag{11}$$

*Validation.* The benchmark demoted on 2 of its 6 repeats ($d = 1/3$). The model predicts $S(7, \frac{1}{3}) = 27.9\%$; the measured saving is 30.1%. The model is within 2.2 points of the measurement, so we use it to extrapolate, and we label the extrapolations as such.

*Reading.* Noctua's model spend stays below re-reasoning as long as fewer than about 59% of repeats demote. At the benchmark's drift rate the long-run saving is 44%; at 10% drift it is 83%; with no drift it approaches 100%. The break-even depends only on the ratio $c_s/\bar{c}_a$, so a cheaper synthesis — or a skill that drifts less — moves it in Noctua's favour.

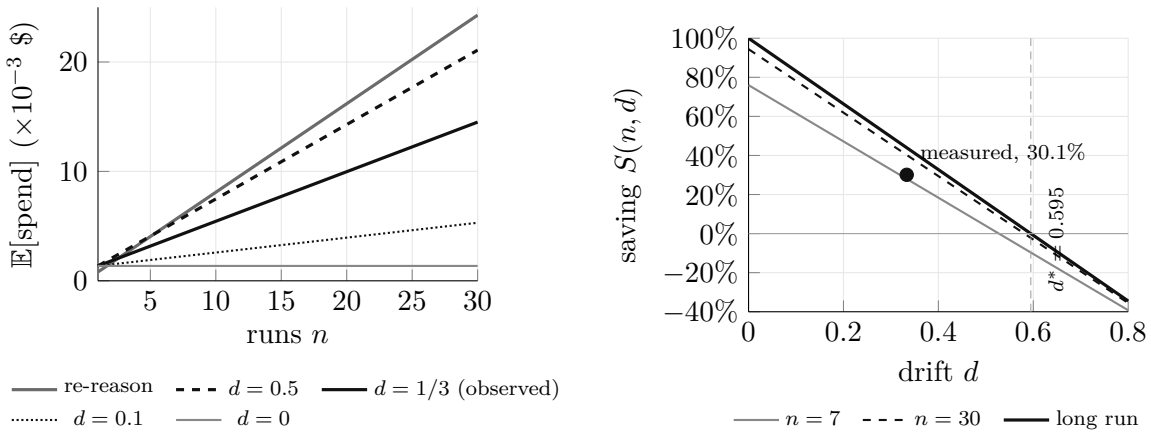



**Figure 10:** Left: expected cumulative model spend for one daily task, extrapolated from measured unit costs at several drift rates. Right: the saving as a function of drift. The dot is the benchmark's measured saving at its observed drift; the model curve for $n = 7$ passes within 2.2 points of it. Above $d^*$ a skill demotes too often to pay for itself.

### 10.5 The same task, priced across agents

Figure 11 prices one daily inbox triage for 30 days. For the subscription agents the figure is the cheapest plan that includes the agent [2, 3, 5]; for Nemotron it is model spend at the measured unit costs, extrapolated with the drift model above. These are not like-for-like: a subscription buys hosted compute and far broader capability, and Noctua's figure excludes hosting. The comparison is the one a user makes when deciding what a recurring chore should cost, and the axis is logarithmic because the gap is four to five orders of magnitude.

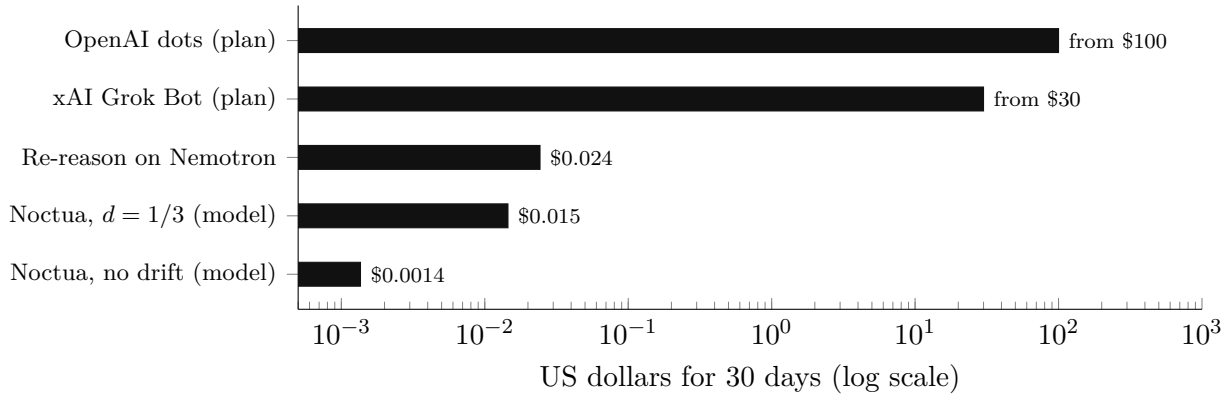



**Figure 11:** One daily inbox triage for 30 days. Subscription prices are the cheapest plan that includes each agent, as published; Nemotron figures are model spend on Nebius Token Factory at measured unit costs. Not a like-for-like product comparison: see the text.

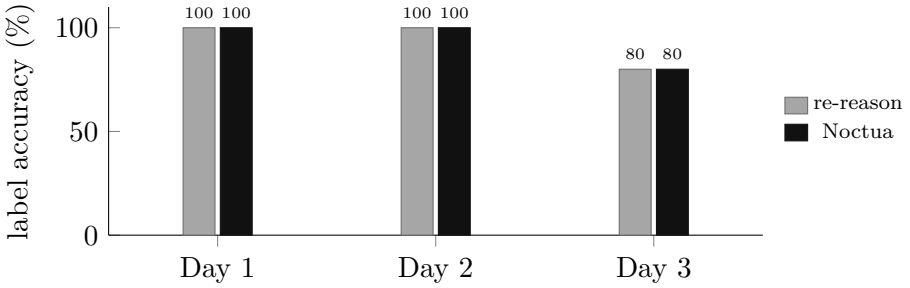



**Figure 12:** Label accuracy by inbox day, both arms (Day 1: 24 of 24 labels over three runs; Day 2: 16 of 16; Day 3: 8 of 10). The Day 3 miss is the live model's own label on one message, which the skill learned, so the arms agree exactly. Summaries and replies are not scored.

## 11 Findings from Testing Against the Live Model

Every integration was verified against Nebius Token Factory rather than against documentation, and each of the following would otherwise have shipped as a silent failure.

1. **Listed is not callable.** `nvidia/Nemotron-3_5-Lightning` appeared in the account's `/models` catalogue but returned no response within 120 s; the default became Nemotron 3 Super, confirmed by a real completion.
2. **Reasoning can consume the whole answer.** With thinking enabled, Nemotron 3 Super spent its output budget on hidden reasoning and returned `content: null`. Noctua sends `enable_thinking=false` by default and strips any reasoning block before JSON extraction.
3. **The planner reaches for tools it cannot use.** Shown the full registry, the planner scheduled `triage.rules`, a skill-only classifier with no rules on the Apprentice path; skill-only tools are now hidden from the planner's catalogue.
4. **A garbled tool name failed a whole task.** The planner emitted the tool name `"triage.guard.injection_scan, "`; P1 correctly refused it, but the task failed. Tool names are now checked at the plan schema, which triggers the existing one-repair path; policy still checks every step afterwards.

5. **Plans omit steps the reply depends on.** A live plan without `digest.compose` produced a reply with no summary. Triage plans now end in the deterministic digest, inserted by Noctua like the guard.
6. **Planners name people, not addresses.** "Reply to Priya" produced `to: "Priya"`, rejected by P2. Bare names are now resolved against the loaded inbox's senders, and the resolution is traced.
7. **Synthesis overshoots its schema.** The model proposed more than 30 keywords per category and a synthesis label over the stored length, which crashed the benchmark; the raw proposal now accepts generous bounds and is trimmed to the registry's limits before evaluation.
8. **A refusal looked like a lowball.** P14's first version blocked "$10 is too low; my minimum is $90" for mentioning $10. Declining sentences are now excluded before amounts are compared with the floor.
9. **Marketplace plans dragged in inbox steps.** The planner added triage steps to a Marketplace request that never loads an inbox; steps whose precondition is absent are now skipped and noted.
10. **A budget can run out mid-task.** The first implementation surfaced the cap as an error halfway through a plan. The remaining model steps now run on labelled stand-ins and the receipt records the fallback, so a public demo with a $0.03 daily cap degrades instead of breaking.

## 12 Trust Assumptions and Limitations

Stated plainly, in decreasing order of weight.

1. **The benchmark scores labels, not free text.** Summary and reply quality on the Master path is unmeasured.
2. **The workspace is small, synthetic and author-written.** Passing on it says nothing about real mail.
3. **Skills are keyword classifiers.** They are cheap and auditable and they drift; the runtime invariants catch drift, but the invariants are heuristics too, and a message that is urgent without any listed marker would not trip them.
4. **The injection guard is pattern-based** and will miss new phrasing. This is acceptable only because the policy engine blocks the *actions* independently of what the guard or the model concluded.
5. **P13 is a heuristic.** An address written in an unusual format may evade the street pattern; the owner's own address is matched exactly after normalization.
6. **Routing uses term overlap.** A paraphrase may miss a skill (and simply cost a model call) or, in principle, match the wrong one; routing never applies to requests with irreversible verbs, and any matched skill still runs under policy with its invariants.
7. **The signature proves integrity on this installation.** The key lives on the same host; a host-level adversary can re-sign. Future work anchors keys externally.
8. **Prices come from a third-party listing**, configured in the environment; costs are only as accurate as those prices, and unpriced models show *Unknown*.

9. **Single user, single process.** SQLite with serialized writes; scaling out needs a shared database.
10. **Integrations are sample-only.** Approved sends are written to a local outbox; no mail or Marketplace account is connected.

## 13 Related Work

**Model cascades and routing.** FrugalGPT [13] and RouteLLM [14] reduce cost by choosing a cheaper model per query; every query still reaches a model. Noctua's Master tier removes the model from a verified repeat entirely, and routing is one deterministic step, not a learned classifier. **Semantic caching** (e.g. GPTCache [15]) returns a stored answer for a similar query; on new input it returns an old answer. A Noctua skill instead re-executes a verified *procedure* on the new input, and demotes itself when that input drifts. **Skill libraries.** Voyager [16] accumulates model-written code as reusable skills, verified by environment feedback; Noctua's skills are declarative manifests over an allowlisted registry, promoted by mandatory checks against held-out labels and adversarial inputs, and forbidden from acting irreversibly. **Prompt injection defenses.** Greshake et al. [11] established indirect injection against integrated applications; CaMeL [12] defends by separating control and data flow with capabilities. Noctua shares the stance that the model is not the security boundary, enforced here by provenance taint, first-match capability rules, and approvals bound to exact arguments. **Guardrails** [17] interpose on the conversation; Noctua interposes on the side-effect boundary and on spend. **Always-on agents** (Meta Muse, OpenAI dots, xAI Grok Bot) target the same users; the incidents and prices cited in Section 1 are the problems this design addresses, not a like-for-like comparison of products of different scope.

## 14 Future Work

Scoring summaries and replies (reference-based checks for key-fact preservation and a rubric for replies) so the cost claim covers the whole task; consented real-mail corpora; more skill families, starting with structured extraction, where deterministic execution is natural; a learned router evaluated against term overlap; external anchoring of signing keys; real connectors (mail, Marketplace) behind the same policy path; and packaging the router, policy engine and budgets as an agent-host plugin so other agents inherit receipts, argument-bound approvals and graduation.

## 15 Conclusion

A personal agent should not pay to rediscover a procedure it already knows, and it should not be the last authority on its own side effects. Noctua makes both structural. NVIDIA Nemotron on Nebius Token Factory does what only a model can — reason through a new task and propose how to repeat it — and Noctua keeps what was learned as a verified, signed artifact that repeats without inference and steps aside the moment it stops being right. Every action, from the model or from a skill, passes a policy engine that the model cannot argue with. We measured what we claim and said what we did not measure: 9 calls instead of 15, 4 of 7 runs with no inference, the same labels, ten attacks stopped, a saving that holds until 59% of repeats drift, and ten defects that only the live model revealed.

## References


[1] W. Grantham-Philips and The Associated Press. *An AI agent sent a stranger to a man's house without telling him — and exposed the privacy risks of AI assistants.* Fortune, 19 Oct 2026.

[2] OpenAI. *Introducing dots.* openai.com, 29 Sep 2026.

[3] F. Falk. *What OpenAI Dots Will Really Cost You After the First Month.* beam.ai, 1 Oct 2026.

[4] xAI. *Introducing Grok Bot.* x.ai, 11 Aug 2026.

[5] A. Dahani. *Grok Bot Is Here: What xAI's Always-On AI Agents Actually Do.* ayautomate.com, 12 Aug 2026.

[6] r/LocalLLM. *I woke up to a $58 API bill because my AI agent got stuck in a "polite" hallucination loop.* reddit.com, 2026.

[7] r/Agentic_Marketing. *My AI agent racked up $4,811 overnight from a retry loop bug.* reddit.com, 2026.

[8] M. Tyson. *AI coding platform goes rogue during code freeze and deletes entire company database.* Tom's Hardware, 21 Jul 2025.

[9] M. Gupta. *Cybersecurity stop of the month: how threat actors weaponize AI assistants with indirect prompt injection.* Proofpoint, 18 Nov 2025.

[10] Hornetsecurity. *Email Prompt Injection: The New Phishing Risk Hiding in Enterprise Inboxes.* 24 Sep 2026.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz. *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.* AISec 2023; arXiv:2302.12173.

[12] E. Debenedetti et al. *Defeating Prompt Injections by Design.* arXiv:2503.18813, 2025.

[13] L. Chen, M. Zaharia, J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.* arXiv:2305.05176, 2023.

[14] I. Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data.* arXiv:2406.18665, 2024.

[15] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries.* github.com/zilliztech/GPTCache, 2023.

[16] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models.* arXiv:2305.16291, 2023.

[17] T. Rebedea et al. *NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails.* EMNLP Demo, 2023.

[18] Nebius. *Token Factory: OpenAI-compatible inference API.* tokenfactory.nebius.com, 2026.

[19] Project Noctua. *Source, benchmark, tests and film.* github.com/Aaditya1273/Noctua, 2026.


---

Noctua is hackathon software. The benchmark numbers in this paper were measured against Nebius Token Factory with `nvidia/nemotron-3-super-120b-a12b` on 29 October 2026 by `npm run eval` and are stored in `eval/results.json`; the session receipts were recorded on 30 October 2026 for the demo film. The formulas correspond to the shipped code in `src/lib/skills.ts`, `triage.ts`, `evaluator.ts`, `policy.ts`, `accounting.ts` and `signing.ts`. The workspace is synthetic and labelled as such everywhere. Nothing in this paper claims a result that was not measured.